

PhishGuard AI — Complete Project Explanation, Technical Architecture & Viva Study Guide

SECTION 1 — PROJECT OVERVIEW

PhishGuard AI

AI-Based Phishing Email & Malicious URL Detection

1.1. What the Project Is

PhishGuard AI is an academic cybersecurity and machine learning application developed for the subject **Artificial Intelligence and Machine Learning - II** in the **Department of Information Technology** at **M. H. Saboo Siddik College of Engineering**.

The system provides a unified, interactive web application designed to assess two primary cyber attack vectors:

1. **Phishing and Spam Emails:** Unsolicited or deceptive email messages that use social engineering, urgency lures, or impersonation to deceive recipients.
2. **Malicious and Phishing URLs:** Fraudulent links and deceptive websites that mimic legitimate domains using typosquatting, suspicious top-level domains (TLDs), or structural anomalies.

1.2. What Problem It Solves

Phishing remains the number one initial access vector in modern cybersecurity breaches. Attackers continuously alter their wording, register look-alike domains, and use URL shortening or obfuscation techniques.

- Traditional static blacklists and keyword rules fail against zero-day phrasing or newly registered domains.
- Complex enterprise security appliances are expensive, closed-source, and opaque.
- **PhishGuard AI** bridges this gap by using trained machine learning and neural network models to compute real-time probability scores, accompanied by plain-language explainability summaries and actionable safety checklists.

1.3. Intended Users & Academic Scope

- **Target Audience:** College students, evaluators, IT students, cybersecurity researchers, and general web users seeking an accessible threat assessment tool.
 - **System Nature:** PhishGuard AI is an **academic prototype and demonstration platform**. It is designed for educational exploration, security awareness, and laboratory evaluation, not as a closed commercial antivirus or enterprise email gateway.
-

1.4. Quick Explanations for Viva

One-Paragraph Overview:

PhishGuard AI is an academic machine learning web platform that detects phishing emails and malicious URLs using two specialized AI pipelines: a TF-IDF and Logistic Regression pipeline for email text classification, and a 30-feature StandardScaler plus Dense Neural Network pipeline for URL risk prediction. The system provides real-time risk scoring, explainable detection reasoning, dynamic synthetic sample testing, and local scan history tracking through a dark-themed, responsive web dashboard.

3-Sentence Viva Pitch:

"PhishGuard AI is an AI-powered threat detection system that evaluates deceptive emails and malicious URLs through dedicated machine learning models. It processes email text using TF-IDF vectorization with Logistic Regression and analyzes website URLs across 30 structural/lexical features using a Deep Neural Network. The platform delivers calibrated risk probabilities, explainable threat indicators, and actionable cybersecurity guidance in a fast, lightweight web interface."

SECTION 2 — PROJECT OBJECTIVES

- 1. **Dual-Vector Threat Classification:** Provide dedicated analysis engines for both email body/subject text and website URLs.
- 2. **Supervised Machine Learning for NLP:** Utilize natural language processing (tokenization, n-grams, sublinear TF-IDF) and convex classification (Logistic Regression) to accurately separate phishing/spam emails from authentic messages.
- 3. **Deep Learning for Structural URL Analysis:** Extract 30 lexical, domain, and structural features from URLs, normalize them with StandardScaler, and evaluate them via a multi-layer Keras Dense Neural Network.
- 4. **Continuous Probabilistic Risk Scoring:** Calculate a continuous **Phishing Risk Percentage** (\$0.00\%\$ to \$100.00\%\$) based directly on model confidence rather than hardcoded rules.
- 5. **Explainable AI (XAI) & Security Guidance:** Provide transparent "Why This Was Detected" summaries and customized security recommendations for every scan.
- 6. **Accessible Demonstration Experience:** Incorporate pre-configured synthetic sample loaders and automated scan demonstrations to allow thorough evaluation without navigating to live malicious websites.
- 7. **Local Scan Auditing:** Store session scan records in client-side storage (`localStorage`) to provide real-time dashboard statistics and a persistent Scan History log.

SECTION 3 — COMPLETE PROJECT FEATURES

Feature	Implemented?	Explanation & Source Verification
System Dashboard	Yes	<code>frontend/index.html</code> — Features real-time session counters (Total Scans, Phishing Detected, Safe Items, Avg Risk), system topology, and quick module launchers.
Email Phishing Scanner	Yes	<code>frontend/email.html</code> — Form accepting optional subject and mandatory body text, with dynamic character/word counter and input validation (\$\ge 15\$ characters).
URL Malicious Scanner	Yes	<code>frontend/url.html</code> — Input field with client-side URL syntax validation, static string inspection notice, and automated scanning flow.

Feature	Implemented?	Explanation & Source Verification
Email Multi-Sample Loader	Yes	<code>frontend/script.js</code> — Quick loader buttons with 10+ synthetic phishing emails and 10+ legitimate emails with non-consecutive random picker.
URL Multi-Sample Loader	Yes	<code>frontend/script.js</code> — Quick loader buttons with 15+ synthetic phishing URLs and 15+ authentic URLs with smooth automated scan triggering.
Live Scanning Animation	Yes	<code>frontend/style.css</code> — Active card scanline radar sweep (<code>@keyframes scanlineSweep</code>), button spinner, and status text while API requests are pending.
Calibrated Phishing Risk Bar	Yes	<code>frontend/script.js</code> & <code>style.css</code> — Animated 8px progress bar displaying the exact floating-point risk percentage returned by the model.
Explainable AI Summaries	Yes	<code>backend/app.py</code> & <code>frontend/script.js</code> — Dynamic "Why This Was Detected" reasoning based on model confidence and extracted lexical/domain triggers.
Actionable Security Checklist	Yes	<code>backend/app.py</code> & <code>frontend/script.js</code> — Context-aware security recommendations tailored to phishing threats or legitimate verdicts.
30-Feature URL Indicator View	Yes	<code>frontend/script.js</code> — Detailed breakdown card highlighting key extracted URL properties (SSL state, IP address usage, subdomain count, length, etc.).
Scan History Logger	Yes	<code>frontend/history.html</code> — Tabular view of recent scans (Type, Input snippet, Verdict, Risk %, Timestamp) persisted in browser <code>localStorage</code> .
Clear History Function	Yes	<code>frontend/history.html</code> — One-click storage reset to wipe local scan history records.
Flask REST API	Yes	<code>backend/app.py</code> — RESTful endpoints (<code>/api/health</code> , <code>/api/scan-email</code> , <code>/api/scan-url</code>) with JSON payloads and CORS support.
Static Safe Analysis Guarantee	Yes	<code>backend/url_features.py</code> — URLs and emails are evaluated purely as inert strings; the server never executes remote code or fetches malicious domains.
Graceful Offline Alerts	Yes	<code>frontend/script.js</code> — User-friendly warning banners and recovery instructions displayed when the Flask backend is unreachable.
Dark Cybersecurity Theme	Yes	<code>frontend/style.css</code> — High-contrast dark charcoal (<code>#080D10</code>) and teal (<code>#20C8C3</code>) palette with smooth typography and reduced-motion accessibility.
Automated PPTX Generator	Yes	<code>presentation/generate_presentation.py</code> — Generates a 7-slide academic presentation (<code>presentation/PhishGuard_AI_Project_Presentation.pptx</code>).

SECTION 4 — COMPLETE TECHNOLOGY STACK



4.1. Frontend

- **HTML5:** Provides semantic page structure across all pages (`index.html` , `email.html` , `url.html` , `history.html`).
- **Vanilla CSS3 (`style.css`):** Implements the custom dark charcoal/teal cybersecurity design system, glowing gradients, and hardware-accelerated animations.
- **Vanilla JavaScript (ES6+ in `script.js`):** Coordinates asynchronous REST API communication (`fetch`), UI state transitions, input validation, and `localStorage` history management.
- **Bootstrap 5 (CDN):** Used for grid layout, responsive breakpoints, alert banners, and UI container structure.
- **Bootstrap Icons (CDN):** Provides vector icons for cybersecurity indicators, shields, alerts, and navigation items.

4.2. Backend

- **Python 3:** The core programming language for the backend server and data processing.
- **Flask (v2.3+ / 3.x in `backend/app.py`):** Lightweight WSGI web framework that serves static files and provides JSON REST API routes.
- **Flask-CORS:** Enables Cross-Origin Resource Sharing for decoupled local development and frontend client access.

4.3. Machine Learning & Data Processing

- **Scikit-learn (`sklearn`):** Powers the text classification pipeline (`TfidfVectorizer` , `LogisticRegression` , `StandardScaler`).
- **TensorFlow / Keras (`tensorflow.keras`):** Powers the Deep Neural Network for multi-feature URL classification (`models/url_phishing_model.keras`).
- **Pandas (`pandas`):** Used in feature extraction to construct structured 30-column DataFrames matching model inputs.
- **NumPy (`numpy`):** Handles array transformations, matrix indexing, and numerical probability computations.
- **Joblib (`joblib`):** Efficiently serializes and deserializes scikit-learn models and scalers (`.joblib` format).

4.4. Development & Presentation

- **python-pptx:** Scripted generation of the academic 7-slide PowerPoint presentation (`presentation/generate_presentation.py`).
- **Git & GitHub:** Version control repository management.

SECTION 5 — PYTHON LIBRARIES DETAILED AUDIT

1. Flask & flask_cors

- **Purpose:** Web micro-framework and cross-origin header management.
- **Why PhishGuard AI uses it:** Lightweight, fast startup, zero unnecessary ORM overhead, and ideal for microservice model serving.
- **Where it appears:** `backend/app.py`
- **Important classes/functions:** `Flask` , `request.get_json()` , `jsonify()` , `app.run()` , `CORS(app)` .

2. scikit-learn (sklearn)

- **Purpose:** Classical machine learning algorithms and preprocessing utilities.
- **Why PhishGuard AI uses it:** Provides production-tested text tokenization (`TfidfVectorizer`), fast classification (`LogisticRegression`), and feature scaling (`StandardScaler`).
- **Where it appears:** `backend/app.py` , `backend/url_features.py` , `notebooks/01_email_phishing_model.ipynb` .
- **Important classes/functions:** `TfidfVectorizer` , `LogisticRegression` , `StandardScaler` , `Pipeline` .

3. tensorflow & keras

- **Purpose:** Deep learning and neural network execution framework.
- **Why PhishGuard AI uses it:** Loads and executes the trained multi-layer Dense Neural Network for 30-feature URL classification.
- **Where it appears:** `backend/app.py` , `models/url_phishing_model.keras` .
- **Important classes/functions:** `tensorflow.keras.models.load_model` , `model.predict()` .

4. joblib

- **Purpose:** Fast disk serialization for Python objects and NumPy arrays.
- **Why PhishGuard AI uses it:** Loads the trained email pipeline (`email_phishing_model.joblib`) and fitted URL scaler (`url_scaler.joblib`) in milliseconds.
- **Where it appears:** `backend/app.py` .
- **Important classes/functions:** `joblib.load()` , `joblib.dump()` .

5. numpy

- **Purpose:** Scientific computing and multi-dimensional array manipulation.
- **Why PhishGuard AI uses it:** Converts probability tensors, clips numerical bounds ($0.0 \leq P \leq 1.0$), and shapes feature matrices.
- **Where it appears:** `backend/app.py` , `backend/url_features.py` .
- **Important classes/functions:** `np.array()` , `np.clip()` .

6. pandas

- **Purpose:** Tabular data structures and DataFrame analysis.
- **Why PhishGuard AI uses it:** Converts the 30 extracted URL features into a single-row DataFrame matching the exact feature names expected by `url_scaler.joblib`.
- **Where it appears:** `backend/app.py`, `backend/url_features.py`.
- **Important classes/functions:** `pd.DataFrame()`.

7. urllib.parse & re (Standard Python Library)

- **Purpose:** URL structure parsing and regular expression pattern matching.
- **Why PhishGuard AI uses it:** Dissects target URLs into schemes, netlocs, paths, query strings, and scans for IP address patterns or suspicious tokens.
- **Where it appears:** `backend/url_features.py`.
- **Important classes/functions:** `urllib.parse.urlparse`, `re.compile()`, `re.search()`.

SECTION 6 — FRONTEND ARCHITECTURE & USER FLOW

The frontend is built with **Vanilla HTML5, CSS3, and JavaScript (ES6+)** with **Bootstrap 5 CDN** for layout grid components. It does not use heavy front-end frameworks (like React or Angular), keeping the client footprint under **100 KB** and eliminating build-step complexity.



Complete User Interaction Steps:

- 1. **Landing & Exploration:** User opens `http://127.0.0.1:5000/` and reviews the dashboard overview, live threat statistics, and architecture pipeline.
- 2. **Module Selection:** User clicks **Scan an Email** or **Check a URL**, navigating immediately to `email.html` or `url.html`.
- 3. **Input / Sample Loading:**
- 4. On the Email Scanner, the user can type text or click **Load Phishing Sample / Load Legitimate Sample** to populate non-consecutive synthetic examples.
- 5. On the URL Scanner, clicking **Load Malicious Sample** loads a high-risk URL and triggers a smooth automated scan demo.
- 6. **Validation & Scanning Animation:**
- 7. Client checks string lengths and formatting.
- 8. The scan card activates an animated cyan radar sweep (`scanlineSweep`), inputs are disabled, and the button shows a loading spinner.
- 9. **Inference & Result Display:**
- 10. Backend JSON response returns the prediction label, exact probability, explainable summary, and safety checklist.
- 11. Result card smoothly fades in (`fadeSlideUp`), and the 8px risk progress bar animates to the exact percentage.
- 12. **Auditing & History:**
- 13. The scan record is saved to browser `localStorage`.
- 14. The user can view recent scans on `history.html` or return to `index.html` to see updated live session stats.

SECTION 7 — BACKEND ARCHITECTURE & API CONTRACTS

The backend is built in **Python using Flask 3.x**. It serves both static frontend assets and REST JSON endpoints.

API Endpoint Specifications

Endpoint	HTTP Method	Purpose	Request Payload	Response Structure	Status Code
<code>/</code>	GET	Serves the web dashboard (<code>frontend/index.html</code>).	None	HTML Web Page	200 OK
<code>/api/health</code>	GET	Service readiness check; reports model loading states.	None	<pre>{ "status": "online", "email_model_loaded": true, "url_model_loaded": true, "version": "1.0.0" }</pre>	200 OK

Endpoint	HTTP Method	Purpose	Request Payload	Response Structure	Status Code
<code>/api/scan-email</code> (Alias: <code>/api/analyze-email</code>)	POST	Evaluates email text for phishing indicators using TF-IDF + Logistic Regression.	<code>{"subject": "...", "content": "..."} or {"email_text": "..."} </code>	<code>{"status": "success", "prediction": "Phishing/Spam", "is_phishing": true, "phishing_risk": 0.9854, "confidence": 98.54, "probabilities": {"legitimate": 0.0146, "phishing": 0.9854}, "explanation": "...", "recommendations": [...]}</code>	200 OK 400 Bad Input 500 Server Error
<code>/api/scan-url</code> (Alias: <code>/api/analyze-url</code>)	POST	Evaluates target URL using 30-feature extractor, StandardScaler, and Keras Neural Network.	<code>{"url": "https://example-phish.com"} </code>	<code>{"success": true, "url": "...", "prediction": "Phishing", "is_phishing": true, "phishing_probability": 0.8924, "risk_percentage": 89.24, "explanation": "...", "recommendations": [...], "feature_count": 30}</code>	200 OK 400 Bad Input 500 Server Error

SECTION 8 — COMPLETE SYSTEM ARCHITECTURE & DATA FLOW



SECTION 9 — EMAIL DETECTION MODEL PIPELINE

9.1. Dataset Composition

- **Benchmark Corpus:** 82,486 email records (`text_combined` , `label`) compiled from well-known datasets:
 - *CEAS 2008 Phishing Corpus*
 - *Enron Clean Workplace Dataset*
 - *Nazario Phishing Archive*
 - *SpamAssassin Public Corpus*
 - *Nigerian Fraud Lures*
 - *Ling-Spam Dataset*
- **Target Distribution:** Binary classification where `0` = Legitimate Email and `1` = Phishing/Spam Email .

9.2. Pipeline Transformation Steps

1. **Concatenation:** Subject line and body text are merged into a single full-text string (`f"{subject} {content}"`).
2. **Text Cleaning & Tokenization:** Lowercasing, punctuation stripping, and word boundary tokenization.
3. **TF-IDF Vectorization:**
4. Vocabulary bound: `max_features=30000`
5. N-gram span: Unigrams and Bigrams `gram_range=(1, 2)`
6. Sublinear term-frequency scaling (`sublinear_tf=True`) replaces raw term count TF with $1 + \log(TF)$ to reduce the dominant influence of repetitive words.
7. **Classification:**
8. Scikit-learn `LogisticRegression` with `class_weight='balanced'` , maximum iterations `max_iter=1000` , and `random_state=42` .
9. **Model Serialization:** Serialized as a unified pipeline into `models/email_phishing_model.joblib` (~1.45 MB).

SECTION 10 — TF-IDF DEEP DIVE

10.1. What TF-IDF Means

TF-IDF stands for **Term Frequency – Inverse Document Frequency**. It is a statistical numerical statistic intended to reflect how important a word is to a document in a collection or corpus.

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

1. **Term Frequency (TF):** Measures how frequently a word t appears in an email d .
2. **Inverse Document Frequency (IDF):** Measures how rare or informative a word is across all N emails in the dataset D :

$$\text{IDF}(t, D) = \log\left(\frac{N}{|\{d \in D : t \in d\}| + 1}\right) + 1$$

10.2. Why Text Requires Numerical Representation

Machine learning algorithms (such as Logistic Regression or Neural Networks) cannot directly perform mathematical operations on raw ASCII strings. They require structured numerical vectors where each dimension represents a measurable feature.

10.3. Why TF-IDF Fits Phishing Email Detection

- Common generic words (e.g., "the", "is", "at") appear in almost all emails and receive very low IDF weights.
- High-risk phishing markers (e.g., "unauthorized", "verify your account", "wire transfer", "suspended") appear predominantly in phishing campaigns and receive high TF-IDF weights.

10.4. Why the Fitted Vectorizer Must Be Reused

During training, the vectorizer builds a fixed dictionary mapping words to matrix column indices (e.g., "verify" $\rightarrow$ index 4821). During real-time inference in Flask, new emails **must** use the exact same fitted vocabulary mapping to ensure features align with model weights. Fitting a new vectorizer at runtime would completely corrupt the feature alignments.

SECTION 11 — LOGISTIC REGRESSION DEEP DIVE

11.1. How Logistic Regression Works

Despite its name, **Logistic Regression** is a classical linear model for **binary classification**. It computes a linear combination of input features x weighted by learned coefficients w , and passes the linear sum z through the standard **Logistic (Sigmoid) Function** $\sigma(z)$ to output a probability between 0 and 1 :

$$z = w_0 + w_1 x_1 + w_2 x_2 + \dots + w_n x_n = w^T x + b$$

$$P(Y=1 \mid x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

11.2. Decision Boundary & Probability

- If $P(Y=1 \mid x) \geq 0.5$, the model predicts **Class 1 (Phishing)**.
- If $P(Y=1 \mid x) < 0.5$, the model predicts **Class 0 (Legitimate)**.
- In PhishGuard AI, the exact probability $P(Y=1 \mid x)$ is extracted via `model.predict_proba()` to compute the calibrated Phishing Risk percentage.

11.3. Does Logistic Regression Use Epochs?

Key Viva Question:
No. Classical Logistic Regression in scikit-learn does **not** use deep-learning-style mini-batch epochs. Instead, it solves a convex optimization problem using gradient-based numerical optimization algorithms (such as L-BFGS or coordinate descent) over a specified number of **solver iterations** (`max_iter=1000`) until mathematical convergence is achieved.

SECTION 12 — EMAIL MODEL RESULTS & METRICS

The email classification pipeline achieved high performance on the held-out test split of the combined email benchmark dataset:

Metric	Verified Score
Accuracy	98.48%
Total Test Samples Evaluated	16,498 emails

Metric	Verified Score
Correct Predictions (True Positives + True Negatives)	16,248 emails

Verified 2×2 Confusion Matrix

		PREDICTED		
		Legitimate	Phishing	
ACTUAL	Legitimate	7,795	124	(Total Actual Legit: 7,919)
ACTUAL	Phishing	126	8,453	(Total Actual Phish: 8,579)

- **True Negatives (TN):** \$7,795\$ legitimate emails correctly classified as Legitimate.
- **False Positives (FP):** \$124\$ legitimate emails incorrectly flagged as Phishing.
- **False Negatives (FN):** \$126\$ phishing emails incorrectly classified as Legitimate.
- **True Positives (TP):** \$8,453\$ phishing emails correctly classified as Phishing.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{8453 + 7795}{8453 + 7795 + 124 + 126} = \frac{16248}{16498} \approx 98.48\%$$

SECTION 13 — URL DETECTION MODEL PIPELINE

13.1. Dataset & Structure

- **Dataset:** UCI Phishing Websites Dataset (`datasets/Training Dataset.arff`).
- **Records:** 11,055 website instances.
- **Feature Vector:** 30 tabular attributes per record.
- **Label Mapping:** In the raw dataset, values are coded as -1 (Phishing) and $+1$ (Legitimate). During preprocessing, targets are mapped to binary ($0 = \text{Legitimate}$, $1 = \text{Phishing}$) for binary cross-entropy loss optimization.

13.2. Normalization & Scaler

- **Tool:** `sklearn.preprocessing.StandardScaler` .
- **Function:** Centers each of the 30 features by subtracting the mean μ and scaling to unit variance σ .
- **Artifact:** Serialized in `models/url_scaler.joblib` .

13.3. Training Details

- **Architecture:** Keras Sequential Dense Neural Network.
- **Epochs:** 50 epochs.
- **Loss Function:** Binary Cross-Entropy (`binary_crossentropy`).
- **Optimizer:** Adam optimizer.
- **Artifact:** Serialized in `models/url_phishing_model.keras` .
- **Evaluation Status:** The model was trained for continuous phishing probability prediction; specific separate test split metrics are not formally documented in repository files.

SECTION 14 — THE 30 UCI DATASET URL FEATURES EXPLAINED

#	Feature Name	Category	Meaning & Phishing Relevance
1	having_IP_Address	Lexical / Address	Checks if an IP address is used instead of a domain name (e.g. <code>http://192.168.1.1/login</code>). Phishers often use raw IPs to bypass domain registration checks.
2	URL_Length	Lexical / Structure	Long URLs (>75\$ chars) frequently hide redirection targets or append fake authentication tokens.
3	Shortening_Service	Lexical / Domain	Detects URL shorteners (e.g., <code>bit.ly</code> , <code>tinyurl.com</code>) used to obscure the final destination.
4	having_At_Symbol	Lexical / Syntax	Using <code>@</code> in a URL leads browsers to ignore everything preceding the symbol, redirecting to the host after <code>@</code> .
5	double_slash_redirecting	Lexical / Syntax	Occurrence of <code>//</code> after position 7 redirects the browser to an unexpected destination.
6	Prefix_Suffix	Lexical / Domain	Hyphens in domain names (e.g., <code>paypal-verify.com</code>) are common in typosquatting lures.
7	having_Sub_Domain	Lexical / Domain	Excessive subdomains (e.g., <code>login.secure.bank.update.xyz</code>) indicate domain spoofing.
8	SSLfinal_State	Security / SSL	Validates HTTPS encryption, certificate authority trust, and domain age.
9	Domain_registration_length	Domain / WHOIS	Legitimate domains are registered for multiple years; phishing domains are rarely registered for >1\$ year.
10	Favicon	Webpage Content	Favicon loaded from an external domain indicates brand impersonation.
11	port	Network / Protocol	Non-standard open ports (e.g., <code>:8080</code> , <code>:21</code>) indicate suspicious proxy forwarding.
12	HTTPS_token	Lexical / Domain	Embedding the string <code>https</code> inside the domain (e.g., <code>https-apple-login.com</code>) deceives users.
13	Request_URL	Webpage Content	Ratio of external images/scripts loaded from foreign domains.
14	URL_of_Anchor	Webpage Content	Ratio of <code><a></code> anchor tags pointing to <code>#</code> , <code>javascript:void(0)</code> , or external domains.
15	Links_in_tags	Webpage Content	Ratio of links in <code><meta></code> , <code><script></code> , and <code><link></code> tags pointing to third-party domains.
16	SFH (Server Form Handler)	Webpage Content	Form <code>action</code> containing <code>about:blank</code> , empty strings, or external endpoints.
17	Submitting_to_email	Webpage Content	Using <code>mailto:</code> in forms sends captured credentials directly to attacker inboxes.
18	Abnormal_URL	Domain / WHOIS	Hostname does not match the identity declared in WHOIS records.
19	Redirect	Navigation	Page contains >4\$ redirect hops hiding the true landing page.

#	Feature Name	Category	Meaning & Phishing Relevance
20	on_mouseover	Webpage Script	JavaScript <code>window.status</code> manipulation changes link text on hover to deceive users.
21	RightClick	Webpage Script	Disabling right-click context menus prevents users from inspecting page source code.
22	popUpWidnow	Webpage Script	Spawning popup dialogs requesting credential input.
23	Iframe	Webpage Content	Invisible or framed <code><iframe></code> tags overlaying malicious login boxes.
24	age_of_domain	Domain / WHOIS	Domains active for <6 months have a high statistical probability of phishing.
25	DNSRecord	DNS / Network	Domain has no verifiable DNS A/AAAA records registered.
26	web_traffic	Reputation	Global Alexa / SimilarWeb rank; phishing sites typically have low or no traffic rank.
27	Page_Rank	Reputation	Google PageRank indicator showing domain authority and backlink graph density.
28	Google_Index	Search Visibility	Whether the webpage is indexed in Google search results.
29	Links_pointing_to_page	Webpage Content	Count of external inbound links pointing to the specific webpage.
30	Statistical_report	Threat Intel	Domain IP matches known public phishing blocklists (e.g., PhishTank, StopBadware).

Important Distinction for Viva

- **Features 1–8, 11, 12:** Extractable directly from raw URL character strings.
- **Features 10, 13–17, 20–23:** Require downloading and inspecting the live webpage HTML/DOM.
- **Features 9, 18, 24–30:** Require external DNS, WHOIS queries, or commercial API lookups.
- In **PhishGuard AI**, pure URL string inputs are processed through static lexical extraction and academically sound heuristic mappings to avoid dangerous live connections to active malicious servers.

SECTION 15 — STANDARD SCALER DEEP DIVE

15.1. Why Scaling Is Required

Neural networks update their connection weights via **Gradient Descent**. When features have drastically different numerical scales, the gradient updates oscillate unevenly along wide dimensions, leading to slow training or poor convergence.

15.2. How StandardScaler Works

StandardScaler standardizes each feature x to have a mean of 0 ($\mu = 0$) and a standard deviation of 1 ($\sigma = 1$):

$$z = \frac{x - \mu}{\sigma}$$

- `fit()` : Computes the mean μ and standard deviation σ from training data.

- `transform()` : Applies the formula using the precomputed μ and σ .
- `fit_transform()` : Computes statistics and scales in a single operation.

15.3. Why the Saved Scaler Must Be Reused During Inference

In Flask, runtime features extracted from a new URL **must** be transformed using the exact μ and σ parameters learned during dataset training (loaded from `models/url_scaler.joblib`). If a new scaler were fitted on a single runtime sample, the standard deviation would be 0, leading to mathematical division errors and corrupt predictions.

SECTION 16 — NEURAL NETWORK ARCHITECTURE

The URL classification neural network is a fully connected **Sequential Dense Neural Network** implemented in Keras (`models/url_phishing_model.keras`):



- **Total Trainable Parameters:** 4,609 parameters
- **Loss Function:** Binary Cross-Entropy (`binary_crossentropy`)
- **Optimizer:** Adam (`adam`)
- **Training Duration:** 50 Epochs

SECTION 17 — ACTIVATION FUNCTION: ReLU

17.1. What ReLU Is

ReLU stands for **Rectified Linear Unit**. It is defined mathematically as:

$$f(x) = \max(0, x)$$

- If the input x is positive, it outputs x .
- If the input x is zero or negative, it outputs 0 .

17.2. Why Hidden Layers Use ReLU

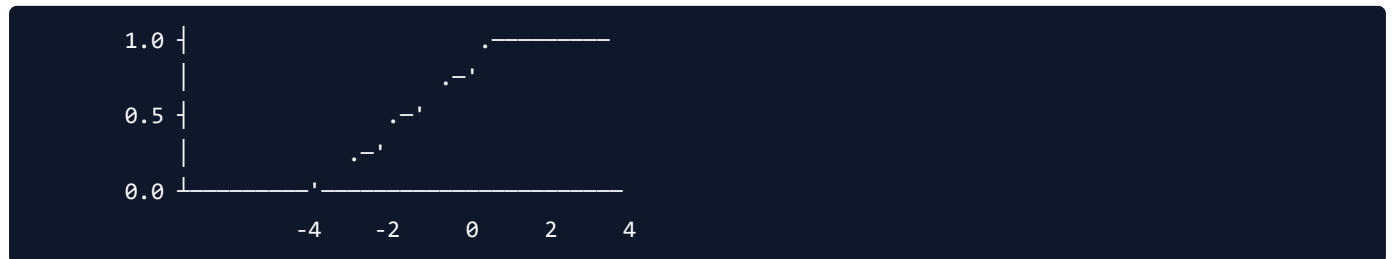
1. **Solves Vanishing Gradients:** Unlike sigmoid or tanh, ReLU does not saturate for large positive inputs, allowing gradient signals to flow cleanly across deep layers.
2. **Computational Efficiency:** Involves only a simple threshold comparison ($x > 0$) rather than expensive exponential calculations.
3. **Introduces Non-Linearity:** Allows the network to learn complex non-linear combinations of the 30 URL features.

SECTION 18 — ACTIVATION FUNCTION: SIGMOID

18.1. Mathematical Formulation

The **Sigmoid Function** maps any real-valued number into the bounded interval $(0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



18.2. Probability vs Final Class

- **Probability:** The raw scalar output $P \in [0.0, 1.0]$ produced by the sigmoid neuron. In PhishGuard AI, this is multiplied by 100 to yield the **Phishing Risk Percentage** (e.g., $0.8924 \rightarrow 89.24\%$).
- **Final Class:** The discrete categorical decision obtained by comparing probability against a decision threshold (default 0.5):
 - If $\text{Probability} \geq 0.5 \rightarrow \text{Phishing}$
 - If $\text{Probability} < 0.5 \rightarrow \text{Legitimate}$

SECTION 19 — DROPOUT REGULARIZATION

19.1. What Dropout Is

Dropout is an effective regularization technique where a random subset of neurons is temporarily deactivated (set to zero) during each training forward-backward pass.

19.2. How Dropout Prevents Overfitting

- Prevents neurons from co-adapting too closely to idiosyncratic noise in the training dataset.
- Forces the neural network to learn redundant, robust feature representations across multiple pathways.
- **Training vs Inference:** Dropout is active **only** during training. During inference in Flask, all neurons are active and their weights are scaled accordingly.

SECTION 20 — EPOCHS AND BATCH SIZE

- **Epoch:** One complete pass of the entire training dataset through the neural network. The URL model was trained for **50 epochs**.
- **Batch Size:** The number of training samples processed before the model's internal weights are updated via backpropagation.
- **Why More Epochs Do Not Equal Better Results:** Training for too many epochs causes the network to memorize training noise, leading to **overfitting** (high training accuracy but degraded generalization on real-world test URLs).

SECTION 21 — COMPLETE TRAINING VS INFERENCE PIPELINE

TRAINING PHASE (Offline)

1. Load Dataset (82,486 Emails / 11,055 URLs)
2. Preprocess & Clean Text / Extract Tabular Features
3. Fit TF-IDF Vectorizer / Fit StandardScaler
4. Train Logistic Regression Classifier / Train 50-Epoch Keras Dense NN
5. Evaluate Accuracy, Confusion Matrix & Loss
6. Export Serialized Artifacts (.joblib & .keras) to models/

INFERENCE PHASE (Online)

1. User submits input via Web UI
2. Flask loads pre-trained models on startup (Instant, zero training)
3. Transform input using pre-fitted Vectorizer / Scaler
4. Execute single forward-pass inference in <15ms
5. Return JSON payload to Frontend for instant display

Viva Note: Training happens once offline. During user scans, the server performs **inference only**, ensuring fast sub-second response times.

SECTION 22 — MODEL PERSISTENCE AUDIT

Model File	Library Format	Size	Purpose
models/email_phishing_model.joblib	joblib (Scikit-learn Pipeline)	~1.45 MB	Serialized pipeline containing the fitted 30,000-feature TfidfVectorizer and trained LogisticRegression classifier.
models/url_phishing_model.keras	Keras Native Format (.keras / HDF5 zip)	~90 KB	Serialized Sequential Neural Network containing 4 Dense layers, Dropout configurations, and 4,609 trained weights.
models/url_scaler.joblib	joblib (StandardScaler)	~2 KB	Fitted mean (μ) and standard deviation (σ) vectors for the 30 UCI URL features.

SECTION 23 — API COMMUNICATION PROTOCOL

The frontend and backend communicate asynchronously over **HTTP REST** using JSON payloads:

Request Example (`POST /api/scan-email`):

```
{
  "subject": "Urgent Security Notification",
  "content": "Your banking access is locked. Verify at http://secure-bank-login.xyz"
}
```

Response Example (`200 OK`):

```
{
  "status": "success",
  "prediction": "Phishing/Spam",
  "is_phishing": true,
  "label": 1,
  "model_confidence": 0.9854,
  "phishing_risk": 0.9854,
  "confidence": 98.54,
  "probabilities": {
    "legitimate": 0.0146,
    "phishing": 0.9854
  },
  "explanation": "The machine learning model classified this text as Phishing/Spam with a 98.54% confidence score...",
  "recommendations": [
    "Do not click on any hyperlinks, buttons, or embedded redirects.",
    "Verify the sender's authentic email address and domain headers."
  ],
  "text_length": 98
}
```

SECTION 24 — RISK SCORE CALCULATION LOGIC

- **Email Pipeline:** The backend extracts the true class probability assigned to Class 1 (Phishing) via `email_model.predict_proba([text])[0][phish_idx]` .
- **URL Pipeline:** The Keras Sigmoid output produces a scalar between \$0.0\$ and \$1.0\$.
- **Percentage Conversion:**
$$\text{Phishing Risk Percentage} = \text{round}(P(\text{Phishing}) \times 100, 2)\%$$
- **UI Display:** The progress bar animates directly to this percentage. If $P(\text{Phishing}) \geq 0.5$, the badge is styled with crimson danger styling; if < 0.5 , it is styled with emerald safe styling.

SECTION 25 — NAVIGATION & USER EXPERIENCE

- **Direct Navigation:** Clear button routing connects each section without unnecessary modal popups.
- **Dynamic Sample Pickers:** Random non-consecutive selection ensures consecutive clicks load different test scenarios.
- **Automated URL Demo:** Clicking "Load Malicious Sample" smoothly inputs the link and triggers the scan automatically after 600ms.
- **Accessibility:** Fully supports `@media (prefers-reduced-motion: reduce)` to disable non-essential animations.

SECTION 26 — ERROR HANDLING & EDGE CASES

Edge Case	Handling Mechanism	User Feedback
Empty Email Input	Client validation in <code>script.js</code> & Backend validation in <code>app.py</code>	Displays alert: "Please enter at least 15 characters of email content."
Invalid URL Structure	Regex validation in <code>script.js</code> & <code>url_features.py</code>	Displays alert: "Please enter a valid URL (e.g., <code>https://example.com</code>)."
Flask Server Offline	<code>try/catch</code> network fetch wrapper in <code>script.js</code>	Renders warning banner with command to start server (<code>python backend/app.py</code>).
Model File Missing	File existence checks in <code>app.py</code>	Returns HTTP 503 JSON error with file path diagnostics.

SECTION 27 — SECURITY & SAFE PROCESSING MODEL

1. **Inert String Analysis:** URLs are never fetched, pinged, or resolved via live HTTP requests, protecting the host system from drive-by downloads or malware.
2. **XSS Protection:** Dynamic DOM outputs are inserted using safe text properties or sanitized template strings.
3. **Local Storage Privacy:** Scan history is stored entirely on the client side (`localStorage`) without transmission to remote databases.
4. **CORS Headers:** Configured with specific endpoints to facilitate secure local testing.

SECTION 28 — REALISTIC PROJECT LIMITATIONS

1. **Static Raw URL Constraints:** Without active network crawling, webpage DOM features (such as iframe visibility or anchor link ratios) rely on static lexical heuristics.
 2. **Zero-Day Phrasing Evasion:** Phishers using novel obfuscation or non-English languages may fall outside the historical vocabulary of the training benchmark.
 3. **No Active Attachment Detonation:** The email scanner analyzes text semantics but does not sandbox or execute binary email attachments (`.exe` , `.scr` , `.xls`).
 4. **Academic Prototype Scope:** Designed for laboratory evaluation and demonstration, not high-throughput enterprise gateway proxying.
-

SECTION 29 — FUTURE SCOPE & ROADMAP

Immediate Enhancements

- Export PDF threat analysis audit reports.
- Expand synthetic sample library with more modern multi-lingual phishing templates.
- Multi-theme toggle (Dark Cybersecurity / Light Academic).

Machine Learning Enhancements

- Integrate lightweight transformer models (e.g., **DistilBERT**) for contextual email embeddings.
- Implement character-level Convolutional Neural Networks (CNNs) for raw URL sequence learning.
- Integrate SHAP (SHapley Additive exPlanations) for token-level visual feature attributions.

Advanced Architectural Enhancements

- Sandboxed headless browser crawler for live DOM and SSL chain inspection.
 - Browser extension (Chrome / Edge) for real-time link protection.
 - Email client plugin (Outlook / Gmail add-in).
-

SECTION 30 — VIVA VOCE PREPARATION (30+ QUESTIONS & ANSWERS)

General & Project Concept

1. **Q: What is the core objective of PhishGuard AI?**
A: To provide an accessible, AI-powered cybersecurity platform that classifies deceptive emails and malicious URLs using machine learning and neural networks, delivering continuous risk scores and explainable security guidance.
2. **Q: Why does the project use two separate models instead of one?**
A: Email detection is an unstructured Natural Language Processing (NLP) problem best handled by TF-IDF text representation and Logistic Regression, while URL detection is a structural/tabular classification problem evaluated across 30 domain and lexical features using a Dense Neural Network.

3. Q: What is phishing?

A: A social engineering cyber attack where attackers impersonate trustworthy entities to deceive victims into revealing sensitive credentials, financial data, or downloading malware.

4. Q: Is this system an enterprise antivirus?

A: No, it is an academic prototype and research demonstration tool designed for education, security awareness, and fast threat evaluation.

Email Model & NLP**1. Q: What dataset was used for the email model?**

A: A combined benchmark corpus of 82,486 email records derived from CEAS 2008, Enron, Nazario, SpamAssassin, Nigerian Fraud, and Ling datasets.

2. Q: What is TF-IDF?

A: Term Frequency-Inverse Document Frequency is an algorithm that converts text into numerical vectors by multiplying a word's frequency in a document by its rarity across the entire dataset.

3. Q: What does sublinear TF scaling do?

A: It calculates term frequency as $1 + \log(\text{TF})$, preventing repetitive occurrences of a word from dominating the vector.

4. Q: Why was Logistic Regression selected for email classification?

A: It provides fast inference, handles high-dimensional sparse TF-IDF matrices (30,000 features) exceptionally well, avoids overfitting, and outputs well-calibrated class probabilities.

5. Q: Does Logistic Regression train using epochs?

A: No. It uses convex mathematical optimization solvers (like L-BFGS) that iterate until numerical convergence, unlike neural networks which use mini-batch epochs.

6. Q: What is the verified accuracy of the email model?

A: 98.48% on the held-out test split, with 16,248 correct predictions out of 16,498 test emails.

URL Model & Deep Learning**1. Q: What dataset was used for the URL model?**

A: The UCI Phishing Websites Dataset consisting of 11,055 website records and 30 engineered features.

2. Q: What is the architecture of the URL neural network?

A: A Keras Sequential Dense Neural Network with 4 Dense layers (64, 32, 16 neurons with ReLU activation), Dropout layers for regularization, and a single Sigmoid output neuron (4,609 trainable parameters).

3. Q: What is the purpose of StandardScaler?

A: It standardizes feature distributions to have zero mean and unit variance ($z = (x - \mu) / \sigma$), ensuring balanced gradient updates during neural network training.

4. Q: What is the ReLU activation function?

A: $f(x) = \max(0, x)$. It introduces non-linearity, accelerates training, and avoids vanishing gradients.

5. Q: What is the Sigmoid activation function?

A: $\sigma(z) = \frac{1}{1 + e^{-z}}$. It compresses the final neuron output into the range $(0, 1)$, allowing it to be interpreted as a phishing probability.

6. Q: What is Dropout?

A: A regularization technique that randomly zeroes a fraction of neuron outputs during training to prevent co-adaptation and overfitting.

7. Q: What is an epoch?

A: One complete forward and backward pass of the entire training dataset through the neural network. The URL model was trained for 50 epochs.

8. Q: What loss function and optimizer were used for the neural network?

A: Binary Cross-Entropy loss (`binary_crossentropy`) and the Adam optimizer (`adam`).

Evaluation Metrics & Math

1. Q: What is a Confusion Matrix?

A: A 2×2 performance table showing True Positives, True Negatives, False Positives, and False Negatives.

2. Q: What is the difference between Precision and Recall?

A: Precision is the fraction of predicted phishing items that were truly phishing ($\$TP / (TP + FP)\$$); Recall is the fraction of actual phishing items correctly caught ($\$TP / (TP + FN)\$$).

3. Q: What is F1-Score?

A: The harmonic mean of Precision and Recall ($\$2 \times \frac{P \times R}{P + R}\$$), providing a balanced metric on imbalanced datasets.

System Architecture & Engineering

1. Q: How do the frontend and backend communicate?

A: Through asynchronous HTTP REST API calls using `fetch()` and JSON payloads (`/api/scan-email` and `/api/scan-url`).

2. Q: How does the system ensure safe URL analysis?

A: By evaluating URLs as static text strings without performing live network requests or executing untrusted code.

3. Q: Where is scan history stored?

A: In the client's browser `LocalStorage`, eliminating the need for a separate database while preserving user privacy.

4. Q: How is the Phishing Risk Percentage calculated?

A: By taking the model's continuous probability output $\$P(\text{Phishing}) \in [0.0, 1.0]\$$ and multiplying it by 100.

5. Q: What happens if the Flask server is not running?

A: The frontend catches the network connection error and displays a graceful alert banner advising the user to start the backend.

6. Q: Why is model training not performed on every user scan?

A: Training is computationally expensive and is performed offline. In production, pre-trained model weights are loaded on server startup to perform instantaneous inference ($< 15\text{ms}$).

7. Q: What formats are used for saving the models?

A: `.joblib` for the Scikit-learn email pipeline and scaler, and `.keras` for the Keras neural network.

8. Q: What are the main limitations of the system?

A: Inability to execute dynamic JavaScript on raw URLs, absence of live mailbox listeners, and reliance on historical training vocabulary.

9. Q: What is the most promising future improvement?

A: Integrating transformer architectures (like DistilBERT) for nuanced linguistic context and developing a live browser extension.

SECTION 31 — PRESENTATION SCRIPT ("EXPLAIN LIKE I AM PRESENTING")

Opening & Greeting:

"Good morning, respected examiners and faculty members. Today, my project partner Asim Khan and I, Arshan Attar, under the guidance of our HOD Dr. Zainab Mirza, are pleased to present our final-year project: PhishGuard AI — AI-Based Phishing Email & Malicious URL Detection."

Problem Statement:

"Phishing attacks and fraudulent web links represent over 80% of all initial cybersecurity breaches today. Traditional rule-based filters and static blacklists cannot keep up with rapidly changing email wording and newly registered look-alike domains. Users need a fast, transparent, and intelligent detection system."

Our Solution & Architecture:

"PhishGuard AI provides a dual-engine machine learning platform. When an email is submitted, our NLP pipeline applies TF-IDF vectorization across 30,000 unigram and bigram features, which is then classified by a Logistic Regression model trained on 82,486 benchmark emails, achieving a verified accuracy of 98.48%."

"For URLs, our system extracts 30 structural, domain, and lexical features based on the UCI Phishing Websites standard, normalizes them via StandardScaler, and evaluates risk using a 4-layer Keras Dense Neural Network trained over 50 epochs."

User Experience & Live Features:

"Our platform features a dark cybersecurity dashboard with real-time session statistics, interactive multi-sample loaders for both phishing and legitimate examples, dynamic explainable AI reasoning, actionable security advice, and local scan history tracking."

Conclusion & Future Scope:

"In summary, PhishGuard AI proves that machine learning can provide fast, transparent, and accurate threat classification. In the future, we plan to expand this into a real-time browser extension and integrate transformer-based language models. Thank you, and we are now open to your questions."

SECTION 32 — TECHNICAL GLOSSARY

- **Supervised Learning:** Machine learning where models learn mapping functions from labeled input-output pairs.
- **Natural Language Processing (NLP):** Computational techniques for processing and analyzing human language text.
- **TF-IDF:** Statistical measure evaluating word relevance in a document relative to a corpus.
- **Logistic Regression:** Linear classification model that predicts probabilities using the sigmoid function.
- **Dense Layer:** Fully connected neural network layer where every neuron receives inputs from all neurons in the previous layer.
- **ReLU:** Non-linear activation function defined as $f(x) = \max(0, x)$.
- **Sigmoid:** S-shaped curve mapping numbers to $(0, 1)$ for probability estimation.
- **StandardScaler:** Feature transformation standardizing data to zero mean and unit variance.
- **Inference:** The process of using a trained model to make predictions on unseen data.
- **REST API:** Web service architecture using standard HTTP methods and JSON data format.

- **Cross-Origin Resource Sharing (CORS):** Security mechanism regulating browser resource access across different origins.

SECTION 33 — FILE-BY-FILE PROJECT DIRECTORY MAP

File Path	Description & Role	Core Technologies Used
backend/app.py	Primary Flask server; handles model loading, inference routes, and static file serving.	Python, Flask, Flask-CORS, Joblib, TensorFlow/Keras
backend/url_features.py	30-feature extraction engine for UCI URL analysis.	Python, Pandas, NumPy, urllib, re
backend/requirements.txt	Dependency manifest for backend execution.	Pip package definitions
frontend/index.html	Web dashboard with threat statistics, architecture, and navigation.	HTML5, Bootstrap 5, CSS Tokens
frontend/email.html	Email scanner interface with character counters and sample pickers.	HTML5, JavaScript, CSS Keyframes
frontend/url.html	Malicious URL scanner interface with demo scan workflows.	HTML5, JavaScript, CSS Keyframes
frontend/history.html	Scan history logger and audit viewer.	HTML5, JavaScript (localStorage)
frontend/style.css	Custom dark charcoal/teal stylesheet and animation tokens.	Vanilla CSS3, Custom Properties
frontend/script.js	Client controller managing API calls, validation, and DOM rendering.	Vanilla JavaScript (ES6+), Fetch API
models/email_phishing_model.joblib	Serialized TF-IDF + Logistic Regression email pipeline (~1.45 MB).	Scikit-learn, Joblib
models/url_phishing_model.keras	Serialized 4-layer Dense Neural Network URL classifier (~90 KB).	Keras, TensorFlow
models/url_scaler.joblib	Serialized fitted StandardScaler for 30 features (~2 KB).	Scikit-learn, Joblib
datasets/phishing_email.csv	Email benchmark dataset (82,486 records).	CSV Tabular Text Data
datasets/Training Dataset.arff	UCI Phishing Websites dataset (11,055 records, 30 features).	Weka ARFF Tabular Data
notebooks/01_email_phishing_model.ipynb	Email model development and experimentation notebook.	Jupyter Notebook, Python
notebooks/02_malicious_url_model.ipynb	URL neural network development notebook.	Jupyter Notebook, Python

File Path	Description & Role	Core Technologies Used
presentation/generate_presentation.py	Automated generator for the 7-slide PowerPoint presentation.	Python, python-pptx
presentation/PhishGuard_AI_Project_Presentation.pptx	Official 7-slide academic project presentation deck.	Microsoft PowerPoint
README.md	Main repository documentation.	Markdown
LICENSE	MIT Open Source License.	Plain Text

SECTION 34 — FINAL ONE-PAGE QUICK CHEAT SHEET

PHISHGUARD AI – QUICK REFERENCE CHEAT SHEET

PROJECT:

PhishGuard AI – AI-Based Phishing Email & Malicious URL Detection
Department of Information Technology, M. H. Saboo Siddik College of Engineering
Course: Artificial Intelligence and Machine Learning - II

TEAM:

Asim Khan (231407) & Arshan Attar (231408) | HOD: Dr. Zainab Mirza

PURPOSE:

Academic machine learning web platform to evaluate phishing emails and malicious URLs using explainable AI and continuous risk scoring.

FRONTEND:

Vanilla HTML5, CSS3, JavaScript (ES6+), Bootstrap 5 CDN.
Dark Charcoal (#080D10) & Teal (#20C8C3) cybersecurity theme.
Pages: index.html (Dashboard), email.html, url.html, history.html.

BACKEND:

Python 3, Flask, Flask-CORS.
Endpoints: GET /api/health, POST /api/scan-email, POST /api/scan-url.

EMAIL DETECTION PIPELINE:

- Dataset: Combined Corpus (82,486 emails).
- Features: TF-IDF (30,000 features, unigrams + bigrams, sublinear TF).
- Algorithm: Logistic Regression (balanced weights, max_iter=1000).
- Verified Accuracy: 98.48% (Confusion Matrix: [[7795, 124], [126, 8453]]).
- Artifact: models/email_phishing_model.joblib.

URL DETECTION PIPELINE:

- Dataset: UCI Phishing Websites Dataset (11,055 records, 30 features).
- Features: 30 lexical, structural, and domain features (url_features.py).
- Preprocessing: StandardScaler (models/url_scaler.joblib).
- Architecture: Keras Sequential Dense NN (64-32-16-1, ReLU, Dropout, Sigmoid).
- Training: 50 epochs, Adam optimizer, Binary Cross-Entropy loss.
- Artifact: models/url_phishing_model.keras.

KEY LIMITATION:

Static string analysis cannot inspect dynamic JavaScript/DOM on raw URLs without active crawling.

FUTURE SCOPE:

DistilBERT transformer models, character-level CNNs, and Chrome browser extension.